Introduction To Algorithms
CS430

Summer 2025

Project Title:
Most Profitable Purchase

Group Members:

1. Andrea Estefania Ramirez-Urena A20570387
2. James Wu Unknown
3. Ikemefuna Onyeka A20611235

Problem Description:

Our assignment for this project is to solve the Most Profitable Purchase problem, in which each item has a price, and if products are purchased in quantity or in combinations, promotions may be used to lower the cost.

The application we designed accepts the following Input files:

1. Items and Quantity file
2. Price of the items file
3. Promotions available file

The program implemented provides the minimum payment for a purchase.

Input files and format

input.txt:
describes merchandise to buy and quantities

```
   <Number of Items>
e.g         2
   <Item_ID_1> <Quantity_1> <Ticket_Price_1>
e.g     9           4               4
   <Item_ID_2> <Quantity_2> <Ticket_Price_2>
e.g     1           5               3
```

promotions.txt:
available promotions with merchandise and amount

```
   <Number of Promotions>
e.g         3
   <Num_Items> <Item_ID_1> <Qty_1> <Promo_Price>
e.g      1           1           4       8
   <Num_Items> <Item_ID_1> <Qty_1> <Item_ID_2> <Qty_2> <Promo_Price>
e.g     2           1           2       2           1       15
   <Num_Items> <Item_ID_1> <Qty_1> <Promo_Price>
e.g     1           9           2       10
```

price.txt:
unit price for merchandise

```
   <Item_ID> <Unit_Price>
e.g     1           3
   <Item_ID> <Unit_Price>
e.g     2           5
   <Item_ID> <Unit_Price>
e.g     9           4
```

Output file format

output.txt:
data showing an optimal purchase plan

```
Total Optimal Cost: $27.00
Promotions Applied:
- [4×1] = $8
Remaining Items:
- 1 × Item 1 @ $3.0 = $3.0
- 4 × Item 9 @ $4.0 = $16.0
```

Data Structures:

Price stores item IDs as keys and unit prices as values in a dict.

```
def parse_prices(path):
    prices = {}
    with open(path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            if len(parts) == 2:
                prices[int(parts[0])] = float(parts[1])
    return prices
```

output example: prices = {1: 3.0, 2: 5.0, 3: 6.0, ...}

Shopping_list stores the quantity of each item the user wants to buy in a dict.

```
def parse_input(path):
    shopping_list = {}
    with open(path, 'r') as f:
        lines = f.readlines()
        count = int(lines[0].strip())
        for line in lines[1:count+1]:
            parts = line.strip().split()
            if len(parts) >= 2:
                shopping_list[int(parts[0])] = int(parts[1])
    return shopping_list
```

output example: shopping_list = {9: 4, 1: 5}

Promotions: each promotion is stored with items and price in a dict.

```
def parse_promotions(path):
    promotions = []
    with open(path, 'r') as f:
        lines = f.readlines()
        num = int(lines[0].strip())
        for line in lines[1:num+1]:
            parts = list(map(int, line.strip().split()))
            if len(parts) < 3:
                continue
            promo = {}
            num_items = parts[0]
            for i in range(num_items):
                item_id = parts[1 + 2*i]
                qty = parts[2 + 2*i]
                promo[item_id] = qty
            promo_price = parts[-1]
            promotions.append({'items': promo, 'price': promo_price})
    return promotions
```

output example: promotions = [
{'items': {1: 4}, 'price': 8},
{'items': {1: 2, 2: 1}, 'price': 15},
]

DP state is stored as a tuple of remaining item quantities, ordered by item_ids because it's hashable and can be used as a dictionary key, which is ideal for caching subproblem results.

```
init_state = tuple(shopping_list[i] for i in item_ids)

memo = {}
def dp(state):
  if state in memo:
  return memo[state]
```

Design & Algorithm:

To determine the most cost-effective combination of items and promotions, the program follows a dynamic programming strategy that systematically explores all valid ways to apply promotions while minimizing the total cost. By evaluating every possible combination of remaining item quantities and valid promotion bundles, and using memoization to avoid redundant calculations, the algorithm ensures that the most profitable combination is selected.

Steps:

1. Convert the shopping list into a tuple of quantities sorted by item IDs so that dynamic programming can be used
2. For each state we try using every applicable promotion, calculate the cost of buying all remaining items at full price and take the minimum of all options.
3. Use a memoization dictionary to cache already computed states for efficiency.

Pseudocode:

```
dp(state):
    if state in memo:
        return memo[state]
    if all quantities are 0:
        return 0
    cost_best = sum of remaining items at unit prices
    for promo in promotions:
        if promo fits into state:
            new_state = state - promo items
            cost_best = min(cost_best, promo.price + dp(new_state))
    memo[state] = cost_best
    return cost_best
```

Performance

The algorithm that we used is a top-down dynamic programming algorithm with memoization (that we call memo in the pseudocode). For every state, we find the optimal subproblem of a smaller state plus its corresponding promotion. Summing up the remaining items and their prices is an O(n) operation, which is repeated p (promotions) times in order to find the best cost. Thus, for one call of dp, the cost at that specific state is O(n * p). There are (q+1)^n possible versions of 'state' because every item in n has at most q+ 1 states (0, 1, … q).

Therefore the time complexity of the entire is: $O(n \cdot p \cdot Q^n)$ where,

n is the number of items.

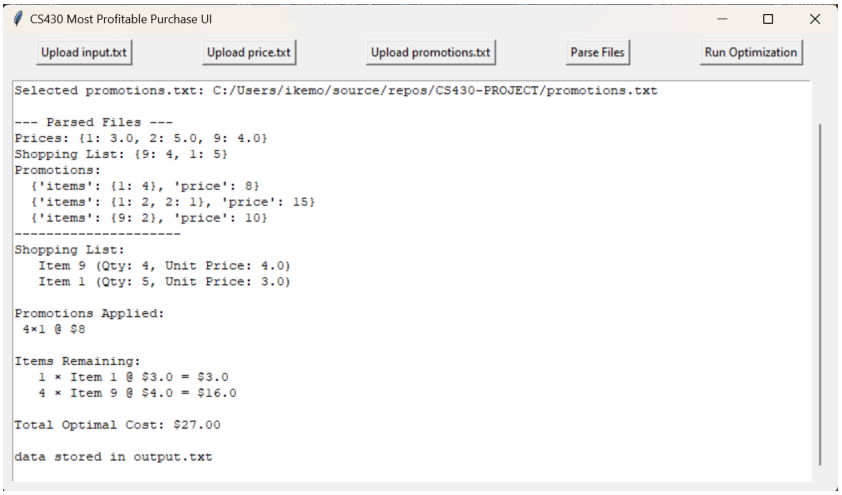
P is the number of promotions

Q^n is the number of possible states

Test cases

Test 1:

input.txt	price.txt	promotions.txt
2	1 3	3
9 4 6	2 5	1 1 4 8
1 5 2.5	9 4	2 1 2 2 1 15
		1 9 2 10

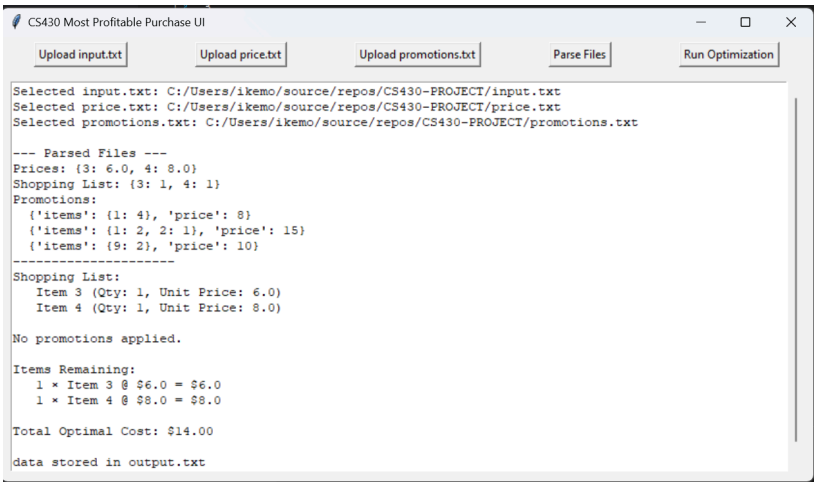


Outcome:

1. Start with a full list: 5 of item 1 and 4 of item 9.
2. Try all valid promotions
3. Apply promotion [4×1 @ $8] since it matches a subset of item 1.
4. Remaining: 1 of item 1 and 4 of item 9.
5. Buy remaining items at full price.

Test 2:

input.txt	price.txt	promotions.txt
2	3 6	3
3 1 6	4 8	1 1 4 8
4 1 8	7 4	2 1 2 2 1 15
		1 9 2 10



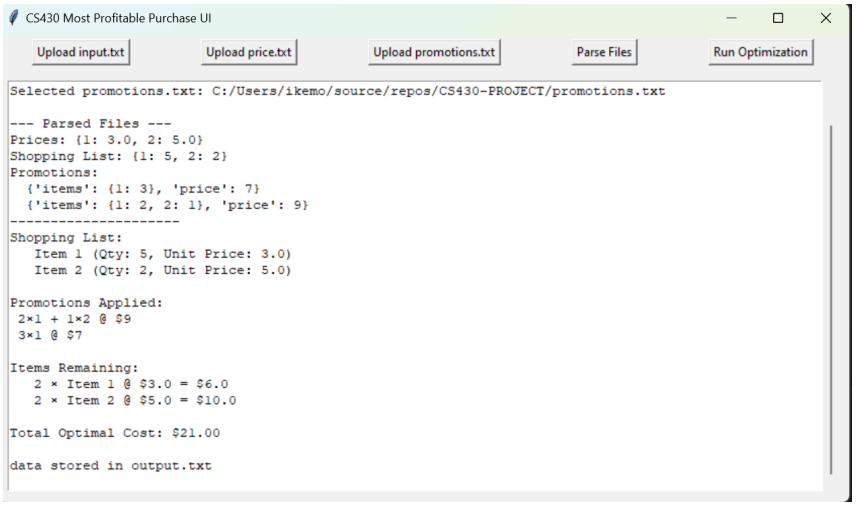
Outcome:

1. Start with a full list: 1 of item 3 and 1 of item 4.
2. Try all valid promotions but Promotions include item 1 and item 9, which do not appear in the shopping list.

3. Buy all items at full price.

Test 3:

input.txt	price.txt	promotions.txt
2	1 3	3
1 5 3	2 5	1 1 3 7
2 2 5		2 1 2 2 1 9



Outcome:

1. Start with a full list: 5 of item 1 and 2 of item 2.
2. Try all valid promotions
3. Apply promotion [3×1 @ $7 and 2x1 + 1x2 @ $9].
4. Buy the remaining 1 of item 2 at full price .

Conclusion

This project combines individual item prices and available discount bundles to provide a well-structured and efficient solution for shopping cost optimization. By employing a recursive dynamic programming technique with memoization to explore every possible combination of item amounts and promotions, the method guarantees that subproblems are avoided and performance remains efficient for modest input volumes.
In terms of functionality, the solution applies the most economical discounts in an intelligent manner while precisely calculating the minimum total cost needed to complete a shopping list. It is useful for practical use cases since it produces a detailed, step-by-step description of the purchase strategy and persistently stores the findings. Overall, the project exhibits careful software design as well as computational control.

To find the auxiliary document you can also go to the ReadMe file on the Github repository..